

document



* Maede Mahmoudi

* 401463169

1) initial data review

 A) Loading datasets and examining their structure :

 `pd.read_csv()` → add csv file to project

 `data.columns` → show columns

 B) Display statistical information :

 `data.isnull().sum()` → number of missing values

 `data['...']` → show specific column

 `data` → show all data

 `Data['...'] = pd.to_numeric(data['...'], errors='coerce')` → convert to float or int, base on data and also errors part is for unvalid data that convert them to NaN

 `data.info()` → information about column's type and memory usage of data

 `numeric_cols = ['age', 'bp', 'bgr', 'bu',
 'sc', 'sod', 'pot', 'hemo', 'pcv', 'wbcc', 'rbcc']`

`for col in numeric_cols:`

`if col in data.columns and data[col].dtype == 'object':`

`data[col] = pd.to_numeric(data[col], errors='coerce')`

`data[numeric_cols].dtypes` → convert object to float type for numeric columns



```
bool_cols = ['ane', 'pe', 'cad', 'dm', 'htn']
```

```
for col in bool_cols:
```

```
    if col in data.columns:
```

```
        data[col] = data[col].astype(bool) → convert boolean columns from object type to bool type
```



```
nominal_cols = ['sg', 'al', 'su', 'rbc', 'pc', 'pcc', 'ba', 'appet', 'class']
```

```
for col in nominal_cols:
```

```
    if col in data.columns:
```

```
        data[col] = data[col].astype('category') → convert to categorical type
```



C) Visual inspection with diagram :



```
data[numeric_cols].hist(figsize=(20, 10), layout=(4, 4), bins=20, color = 'orange')
```

```
plt.tight_layout() → histogram plot for numeric columns and the next histplot is for hemoglobin
```



```
sns.histplot(data=data, x='age', hue='class', multiple='stack', palette='viridis')
```

```
plt.title('Age distribution based on disease class') → histplot for age column based on class column
```

🍓 `sns.heatmap(data.isnull(), cbar=False, cmap='viridis')` → heatmap for missing value

🍓 `corr_matrix = data.select_dtypes(include=['float64', 'Int64']).corr()` → heatmap for numeric column, correlation between numeric attr, The bolder the more cohesive

🍓 2) Management of missing values

🍓 A) Identify missing values :

🍓 `data = data.replace('?', np.nan)` → replace ? With Nan

🍓 `data.to_csv('data.csv', index=False, na_rep='Nan')` → update csv file

🍓 `binary_cols = ['htn', 'dm', 'pe', 'ane', 'cad']`

for col in binary_cols:

if col in data.columns:

`data[col] = data[col].replace({True: 1, False: 0})` → convert true and false in columns to 1 & 0

🍓 B) Remove samples with many missing values :

🍓 `threshold_percentage = 0.4`

`threshold_value = len(data) * threshold_percentage`

`nan_counts = data.isna().sum()`

`columns_to_drop = nan_counts[nan_counts > threshold_value].index`

`cleaned_data = data.drop(columns = columns_to_drop)` → First, we set the threshold value to 40% and multiply the result by the number of rows, then calculate the number of missing values and then remove them.

C) Replacing missing values with the mean for numeric attr :

 `numeric_cols = ['age', 'bp', 'bgr', 'bu', 'sc', 'sod', 'pot', 'hemo', 'pcv', 'wbcc', 'rbcc']`

`for col in numeric_cols:`

 `data[col].fillna(round(data[col].median() , 3), inplace=True)` → replacing missing values with mean but I decided to round them to three number after dot, Because when I didn't specify how many decimal places it had, my number had lot of decimal point after it

D) Replacing missing values with the mean for nominal attr :

 `nominal_cols = ['sg', 'al', 'su', 'rbc', 'pc', 'pcc', 'ba', 'htn', 'dm', 'cad', 'appet', 'pe', 'ane', 'class']`

`data = data.copy()`

`for col in nominal_cols:`

`if col in data.columns:`

`mode_value = data[col].mode()[0]`

`data[col] = data[col].fillna(mode_value)` → replacing missing value for nominal attribute

🍓 3) Identify and remove outliers

🍓 A) using IRQ :

🍓 `numeric_cols = ['age' , 'bp' , 'bgr' , 'bu' , 'sc' , 'sod' , 'pot' , 'hemo' , 'pcv' , 'wbcc' , 'rbcc']`

`for col in numeric_cols :`

`if col in data.columns:`

`q1 = data[col].quantile(0.25)`

`q3 = data[col].quantile(0.75)`

`irq = q3 - q1`

`lower_bound = q1 - 1.5*irq`

`upper_bound = q3 + 1.5*irq`

`data = data[(data[col] >= lower_bound) & (data[col] <= upper_bound)]` →for numeric attr I calculate IRQ, first we should identify first quantile and third, Any data that is less than or greater than 1.5 times is outlier data

🍓 B) outliers with boxplot :

🍓 `sns.boxplot(data = data[numeric_cols])`

🍓 C) Replace outliers if necessary :

🍓 There isn't necessary

4) convert & normalized data

A) normalized with min_max scaling :

```
scaler = MinMaxScaler(feature_range=(0, 1))
```

```
data[numeric_cols] = scaler.fit_transform(data[numeric_cols]) → normalized in range [0,1]
```

5) Converting numeric features to categorical

A) label encoding for nominal attr :

```
 encoder = LabelEncoder()
```

```
data['rbc_encoded'] = encoder.fit_transform(data['rbc'])
```

```
data['pc_encoded'] = encoder.fit_transform(data['pc'])
```

```
data['pcc_encoded'] = encoder.fit_transform(data['pcc'])
```

```
data['ba_encoded'] = encoder.fit_transform(data['ba'])
```

```
data['appet_encoded'] = encoder.fit_transform(data['appet'])
```

```
data['class_encoded'] = encoder.fit_transform(data['class'])
```


🍓 B) one Hot encoding :

```
🍓 data_encoded = pd.get_dummies(  
    data,  
    columns = nominal_cols,  
    prefix = nominal_cols,  
    drop_first = True → I write this by chat gpt help :)  
)
```

🍓 6) Correlation analysis and dimensionality reduction :

🍓 A) Correlation check :

```
🍓 numeric_data = data.select_dtypes(include=['number'])  
correlation_matrix = numeric_data.corr()  
  
sns.heatmap(correlation_matrix, annot=True, cmap="coolwarm", fmt='.2f' → heat map to show correlation between numeric data
```

🍓 `pca = PCA(n_components=2)`

`pca_data = pca.fit_transform(data[numeric_cols])`

`pca.explained_variance_ratio_` → We want to reduce the data to two components, last line This value represents each principal component of the total variance of the data

- 🍓 B) using pca for reduction dimensions :
- 🍓 `pca_df = pd.DataFrame(data=pca_data, columns=['PC1', 'PC2'])` → data frame
- 🍓 `plt.scatter(pca_data[:, 0], pca_data[:, 1], c=colors, cmap='viridis', edgecolor='k', s=50)` → scatter plot for it